JOBSHEET 8
PERULANGAN 2

1. Tujuan

- Memahami konsep perulangan bersarang pada algoritma
- Memahami alur perulangan bersarang
- Mengenal sintaks perulangan bersarang
- Mampu menerapkan perulangan bersarang untuk menyelesaikan studi kasus sederhana.

2. Praktikum

2.1 Percobaan 1: Review Perulangan 1

Waktu Percobaan: 30 menit

Percobaan ini mengulang konsep perulangan FOR sebelum mempelajari perulangan bersarang. Program menampilkan karakter `*` sebanyak *n* kali secara mendatar (ke arah samping). Gunakan input *n* berupa bilangan bulat positif.

Langkah-langkah Percobaan

1. Buat repository baru bernama **daspro-jobsheet8** di GitHub, *clone* ke komputer Anda, lalu buka foldernya di Visual Studio Code.
2. Buat file baru **StarNoAbsen.java** di dalam folder tersebut. Ganti NoAbsen dengan nomor absen Anda, misalnya **Star05.java**.
3. Tambahkan sintaks import class `Scanner` pada baris paling atas program.

```
import java.util.Scanner;
```

4. Buat class dan fungsi/method `main()`.

```
public class StarNoAbsen {
    public static void main(String[] args) {

    }
}
```

5. Di dalam method `main()`, deklarasikan objek `Scanner` dengan nama `sc`, kemudian tampilkan instruksi untuk memasukkan nilai yang akan disimpan ke variabel `n`.

```
Scanner sc = new Scanner(System.in);
System.out.print("Masukkan nilai n = ");
int n = sc.nextInt();
```

6. Pada baris selanjutnya, buat perulangan FOR untuk mencetak `*` sebanyak *n* kali. Perhatikan bahwa yang digunakan adalah `print`, **bukan** `println`, karena karakter `*` ingin ditampilkan berderet tanpa pindah baris.

```
for (int i = 1; i <= n; i++) {
    System.out.print("*");
}
```

7. Setelah perulangan selesai, tambahkan `System.out.println();` agar kursor pindah ke baris baru, lalu tutup objek Scanner dengan `sc.close();`. Kode lengkap program menjadi seperti berikut.

```
import java.util.Scanner;

public class StarNoAbsen {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
        sc.close();
    }
}
```

8. Simpan file, kemudian *compile* dan jalankan program.
9. Masukkan n = 5. Amati hasilnya; output harus serupa dengan tampilan berikut (teks tebal biru adalah input yang diketik melalui *keyboard*).

```
Masukkan nilai n = 5
*****
```

10. Jalankan kembali program dengan n = 3 dan n = 7. Catat jumlah karakter `*` yang muncul pada setiap pengujian.
11. Setelah program berjalan sesuai harapan, *commit* dan *push* file **StarNoAbsen.java** ke repository **daspro-jobsheet8**.

Pertanyaan

1. Jika inisialisasi `i = 1` diubah menjadi `i = 0` tanpa mengubah kondisi, berapa karakter `*` yang tercetak ketika n = 5? Jelaskan.
2. Jika kondisi `i <= n` diganti menjadi `i < n`, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

```
for (int i = 2; i <= n; i += 2) {
    System.out.print(i + " ");
}
```

   Modifikasi kode tersebut agar mencetak bilangan **ganjil** positif yang tidak melebihi n.
4. Jelaskan mengapa perubahan *increment* `i++` menjadi `i--` (dengan kondisi tetap `i <= n`) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan **Ctrl + C** pada terminal.

2.2 Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)

Waktu Percobaan: 30 menit

Pada percobaan ini digunakan *nested loop* FOR untuk mencetak persegi panjang bintang. Jumlah baris dan kolom dimasukkan secara terpisah sehingga peran kedua loop dapat diamati dengan jelas: *outer loop* mengatur **baris**, sedangkan *inner loop* mengatur **kolom** (ingat analogi rak/loker pada materi).

Contoh input baris = 3 dan kolom = 6 menghasilkan output:

```
******
******
******
```

Langkah-langkah Percobaan

1. Buat file baru **RectangleNoAbsen.java**.
2. Tambahkan `import java.util.Scanner;`, kemudian buat class `RectangleNoAbsen` beserta method `main()` seperti pada Percobaan 1.
3. Di dalam `main()`, deklarasikan Scanner `sc`, lalu baca dua bilangan bulat positif, yaitu `baris` dan `kolom`.

```
Scanner sc = new Scanner(System.in);
System.out.print("Masukkan jumlah baris: ");
int baris = sc.nextInt();
System.out.print("Masukkan jumlah kolom: ");
int kolom = sc.nextInt();
```

4. Tuliskan terlebih dahulu perulangan yang mencetak `*` sebanyak `kolom` kali (sama seperti Percobaan 1). Perulangan ini nantinya menjadi *inner loop*.

```
for (int j = 1; j <= kolom; j++) {
    System.out.print("*");
}
```

5. Jalankan program dengan input baris = 3 dan kolom = 6. Perhatikan bahwa yang tercetak baru **satu baris** bintang, karena belum ada perulangan untuk baris.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
```

6. Bungkus perulangan tersebut dengan *outer loop* yang berulang sebanyak `baris` kali. Tambahkan `System.out.println();` **setelah** *inner loop* (masih di dalam *outer loop*) agar setiap baris bintang dicetak pada baris baru.

```
for (int i = 1; i <= baris; i++) {        // outer loop: baris
    for (int j = 1; j <= kolom; j++) {    // inner loop: kolom
        System.out.print("*");
    }
    System.out.println();                 // pindah baris
}
```

7. Tutup Scanner dengan `sc.close();` di akhir method `main()`. Kode lengkap program menjadi seperti berikut.

```
import java.util.Scanner;

public class RectangleNoAbsen {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();
        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

8. Simpan, *compile*, dan jalankan program dengan input baris = 3 dan kolom = 6. Output harus serupa dengan tampilan berikut.

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
```

9. Jalankan kembali untuk input (baris, kolom) = (4, 2) dan (1, 5). Bandingkan bentuk ketiga output tersebut.

Pertanyaan

1. Tentukan fungsi variabel `i` dan `j` pada program tersebut.
2. Berapa kali *outer loop*, *inner loop*, dan perintah `print("*")` dijalankan untuk baris = 4 dan kolom = 7?
3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan `kolom`, inner menggunakan `baris`)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.
4. Jelaskan akibat penghapusan `System.out.println();` setelah *inner loop*.
5. Modifikasi program agar karakter yang dicetak adalah `#`, lalu *commit* dan *push* perubahan ke GitHub.

2.3 Percobaan 3: Debugging Bintang Segitiga – Nested WHILE

Waktu Percobaan: 45 menit

Kode awal pada percobaan ini sengaja dibuat belum menghasilkan format segitiga yang benar. Tugas Anda adalah mengidentifikasi kesalahannya dan memperbaikinya tanpa mengganti struktur WHILE menjadi FOR.

Output target untuk n = 5:

```
*
**
***
****
*****
```

Langkah-langkah Percobaan

1. Buat file baru **TriangleNoAbsen.java**.
2. Ketik kode awal berikut secara lengkap (sesuaikan nama *class* dengan nama file Anda).

```java
import java.util.Scanner;

public class TriangleNoAbsen {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        int i = 0;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            i++;
        }
        sc.close();
    }
}
```

3. Simpan dan jalankan program dengan n = 3, kemudian dengan n = 5. Catat output aktual yang muncul, lalu bandingkan dengan output target.
4. Untuk mencari penyebab kesalahan, tambahkan perintah bantu (debug) sementara tepat setelah baris `while (i <= n) {`. Perintah ini menampilkan nilai `i` setiap kali *outer loop* dimulai.

```java
while (i <= n) {
    System.out.print("[i=" + i + "] ");   // debug sementara
    int j = 0;
```

5. Jalankan kembali program dengan n = 5, lalu amati hasilnya:
   a. berapa kali *outer loop* dijalankan? Apakah sama dengan jumlah baris yang diinginkan?
   b. berapa karakter `*` yang tercetak saat `i = 0`?
   c. mengapa semua karakter tampil dalam satu baris?
6. Perbaiki kode satu per satu dan jalankan program setelah setiap perbaikan:
   a. tambahkan `System.out.println();` setelah *inner loop* selesai, sebelum `i++`. Jalankan dan amati: apakah masih ada baris yang tidak berisi `*`?

b. ubah inisialisasi dan/atau kondisi *outer loop* agar perulangan berjalan tepat *n* kali tanpa baris kosong. Jalankan kembali.

7. Setelah output sama dengan output target, **hapus** baris *debug*. Jalankan sekali lagi untuk memastikan hasilnya, lalu *commit* dan *push* ke repository Anda.
8. Modifikasi program agar **setelah** segitiga di atas tercetak, program juga mencetak segitiga terbalik. Tuliskan nested WHILE kedua di bawah nested WHILE pertama (masih di dalam `main()`), tanpa mengganti WHILE menjadi FOR.

> **Catatan:** variabel `i` sudah dideklarasikan sebelumnya, jadi pada *nested* WHILE kedua cukup tulis `i = n`; tanpa `int`. Menulis `int i = n;` akan menyebabkan *error*: *variable i is already defined*.

9. Jalankan program dengan n = 5. Bagian segitiga terbalik harus serupa dengan tampilan berikut.

```
*****
****
***
**
*
```

Pertanyaan

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
2. Jelaskan peran variabel `i` dan `j`. Mengapa nilai `j` harus diinisialisasi kembali ke 0 pada setiap iterasi *outer loop*? Apa yang terjadi jika `j` tidak di-*reset*?
3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (*update*) pada *outer loop* yang diperlukan untuk membentuk segitiga terbalik.

2.4 Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok

Waktu Percobaan: 75 menit

Dalam penilaian proyek kelompok, setiap kelompok memperoleh nilai dari sejumlah penilai. Program menerima jumlah kelompok dan jumlah penilai yang sama untuk setiap kelompok, kemudian menghitung rata-rata nilai tiap kelompok dan menentukan kelompok dengan rata-rata tertinggi. Kasus ini diselesaikan dengan *nested loop* tanpa perlu menyimpan seluruh nilai: **WHILE** (luar) untuk kelompok, **FOR** (tengah) untuk penilai, dan **DO-WHILE** (dalam) untuk mengulang input nilai yang tidak valid.

Langkah-langkah Percobaan

1. Buat file baru **NilaiKelompokNoAbsen.java** yang berisi `import java.util.Scanner;`, class, method `main()`, dan deklarasi Scanner dengan nama `sc`.
2. Minta input jumlah kelompok dan jumlah penilai. Keduanya harus lebih besar dari 0; gunakan IF-ELSE sehingga seluruh proses penilaian hanya dijalankan pada blok `else`.

```
System.out.print("Masukkan jumlah kelompok : ");
int jumlahKelompok = sc.nextInt();
System.out.print("Masukkan jumlah penilai  : ");
int jumlahPenilai = sc.nextInt();

if (jumlahKelompok <= 0 || jumlahPenilai <= 0) {
    System.out.println("Jumlah kelompok dan penilai harus > 0.");
} else {
    // langkah 3 sampai 8 ditulis di dalam blok else ini
}
sc.close();
```

3. Di dalam blok `else`, siapkan variabel yang nilainya harus **bertahan** selama semua kelompok diproses.

```
int nomorKelompok = 1;        // penghitung outer loop
double rataTertinggi = -1;    // -1 agar kelompok pertama pasti tercatat
int kelompokTerbaik = 0;
```

4. Buat *outer loop* menggunakan WHILE untuk memproses kelompok satu per satu. Pada awal setiap kelompok, tampilkan nomor kelompok dan *reset* `totalNilai` menjadi 0. Jangan lupa `nomorKelompok++` di akhir badan perulangan; tanpa baris ini perulangan tidak akan berhenti.

```
while (nomorKelompok <= jumlahKelompok) {
    System.out.println("Kelompok " + nomorKelompok);
    double totalNilai = 0;

    // inner loop penilai (langkah 5)

    // hitung rata-rata dan cek kelompok terbaik (langkah 6-7)

    nomorKelompok++;
}
```

5. Ganti komentar *inner loop penilai* dengan perulangan FOR untuk memasukkan nilai dari setiap penilai dan menjumlahkannya ke `totalNilai`. Nilai valid berada pada rentang 0–100; gunakan DO-WHILE agar input nilai yang sama diminta kembali selama nilainya di luar rentang.

```
for (int j = 1; j <= jumlahPenilai; j++) {
    int nilai;
    do {
        System.out.print("  Nilai dari Penilai " + j + ": ");
        nilai = sc.nextInt();
        if (nilai < 0 || nilai > 100) {
            System.out.println("  Nilai harus 0-100. Ulangi input!");
        }
    } while (nilai < 0 || nilai > 100);
    totalNilai += nilai;
}
```

6. Setelah *inner loop* selesai, hitung rata-rata menggunakan tipe `double` dan tampilkan hasilnya.

```
double rataNilai = totalNilai / jumlahPenilai;
System.out.println("Kelompok " + nomorKelompok
        + ": nilai rata-rata = " + rataNilai);
```

7. Bandingkan `rataNilai` dengan `rataTertinggi`. Lengkapi sendiri isi blok IF berikut. Jika rata-rata sama, kelompok yang ditemukan lebih awal tetap menjadi pilihan (karena digunakan operator `>`, bukan `>=`).

```
if (rataNilai > rataTertinggi) {
    // lengkapi: perbarui rataTertinggi dan kelompokTerbaik
}
```

8. Setelah kurung kurawal penutup WHILE (masih di dalam blok `else`), tampilkan kelompok terbaik dan rata-rata tertinggi.

```
System.out.println("==============================");
System.out.println("Kelompok terbaik    : " + kelompokTerbaik);
System.out.println("Rata-rata tertinggi : " + rataTertinggi);
```

9. Simpan, *compile*, dan jalankan program dengan 3 kelompok dan 2 penilai. Uji juga validasi dengan sengaja memasukkan nilai 105. Output harus serupa dengan tampilan berikut.

```
Masukkan jumlah kelompok : 3
Masukkan jumlah penilai  : 2
Kelompok 1
  Nilai dari Penilai 1: 80
  Nilai dari Penilai 2: 90
Kelompok 1: nilai rata-rata = 85.0
Kelompok 2
  Nilai dari Penilai 1: 75
  Nilai dari Penilai 2: 105
  Nilai harus 0-100. Ulangi input!
  Nilai dari Penilai 2: 85
Kelompok 2: nilai rata-rata = 80.0
Kelompok 3
  Nilai dari Penilai 1: 90
  Nilai dari Penilai 2: 96
Kelompok 3: nilai rata-rata = 93.0
==============================
Kelompok terbaik    : 3
Rata-rata tertinggi : 93.0
```

10. Jalankan kembali dengan variasi lain, misalnya 2 kelompok dan 3 penilai, serta uji input tidak valid (jumlah kelompok = 0) untuk memastikan pesan kesalahan tampil.
11. *Commit* dan *push* kode program ke GitHub.

Pertanyaan

1. Mengapa `totalNilai` perlu di-*reset* di dalam *outer loop*, tetapi `rataTertinggi` tidak boleh di-*reset* pada setiap kelompok?

2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
3. Apa yang terjadi jika `totalNilai` dideklarasikan sebagai `int` dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
4. Mengapa `rataTertinggi` diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

3. Tugas

Waktu Pengerjaan: 120 menit

Kerjakan tugas berikut secara mandiri. Gunakan *nested loop*, struktur pemilihan jika diperlukan, dan cukup gunakan variabel biasa (variabel penampung dan penghitung) untuk menyimpan hasil perhitungan. Buat file terpisah untuk setiap tugas. *Commit* dan *push* seluruh file ke repository **daspro-jobsheet8** di GitHub.

1. Tugas 1: Pola Bilangan Bertingkat

Buat program untuk mencetak pola bilangan 1 sampai nomor baris. Masukkan bilangan bulat n (n minimal 1). Pada baris pertama tercetak 1, pada baris kedua 1 2, dan seterusnya sampai baris ke-n. Setelah pola selesai, tampilkan total banyaknya angka yang telah dicetak. Simpan dengan nama file: **Tugas1NoAbsen.java**.

Contoh untuk n = 5:

```
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

2. Tugas 2: Tabel Perkalian

Buat program untuk menampilkan tabel perkalian berukuran n × n menggunakan nested FOR atau WHILE. Nilai n minimal 3. Setiap posisi menampilkan hasil perkalian nomor baris dan nomor kolom. Atur spasi agar keluaran rapi, lalu tampilkan jumlah seluruh angka pada tabel. Simpan program dengan nama file: **Tugas2NoAbsen.java**.

Contoh untuk n = 4:

```
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
```

Petunjuk: agar angka tersusun rata kanan, gunakan `System.out.printf("%4d", hasil);` yang mencetak bilangan dengan lebar 4 karakter, atau gunakan karakter tab `"\t"` sebagai pemisah.

3. Tugas 3: Rekap Penjualan Beberapa Toko

Sebuah pusat perbelanjaan memiliki beberapa toko. Setiap toko melayani sejumlah pelanggan pada hari yang sama, dan setiap pelanggan membeli beberapa barang. Program meminta jumlah toko, kemudian jumlah pelanggan pada masing-masing toko, dan jumlah barang yang dibeli setiap pelanggan. Hitung dan tampilkan total pelanggan serta total barang per toko, total barang seluruh toko, dan nomor toko dengan jumlah barang terbanyak. Jika terdapat nilai tertinggi yang sama, tampilkan toko dengan nomor paling kecil. Simpan program dengan nama file: **Tugas3NoAbsen.java.**

Ketentuan: jumlah toko minimal 1; jumlah pelanggan dan jumlah barang setiap pelanggan boleh 0, tetapi tidak boleh negatif. Gunakan perulangan bersarang; data tidak perlu disimpan satu per satu, cukup diakumulasikan ke variabel total.

Contoh: Toko 1 memiliki 2 pelanggan yang membeli 3 dan 2 barang; Toko 2 memiliki 3 pelanggan yang membeli 1, 4, dan 2 barang. Ringkasan yang diharapkan:

```
Toko 1 - pelanggan: 2, total barang: 5
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
```